Movie Performance Management System

Architecture Blueprint

Group Report

Problem and Glossary | Event Storming | C4 Architecture | Team Topologies | Shared Rules

October 2026

Team members	**Modules**
Prakash	Input page and report page
Rose	Database
Chandani	API
Reinaldo	UI/UX and Git administration
Avishek (Avi)	Data handling and visuals

Abstract

This report documents the architecture blueprint for the Movie Performance Management System, a web-based prototype that helps cinema employees record the money each movie earns per week, read a simple weekly report, and identify which movies need management review. The report defines the problem and a shared glossary, presents an event storming board that brings out the domain events, commands and hot spots, and describes the system with the first three levels of the C4 model, including the web application, the REST API and the relational data store that communicate through REST and JSON. It then maps the five-member team to the Team Topologies model and sets out shared rules for error handling and authentication. Together with the OpenAPI 3.0 contract (`openapi.yaml`), these decisions let each module be built independently while remaining compatible.

Table of Contents

1. Problem and Glossary 1
1.1 Purpose of This Document 1
1.2 Problem Statement 1
1.3 Scope and Intended Users 1
1.4 Glossary 2
2. Event Storming Board 4
2.1 Approach 4
2.2 The Event Storming Board 4
2.3 Events, Commands and Aggregates 5
2.4 Hot Spots and Decisions 6
2.5 Bounded Contexts 7
3. C4 Architecture Diagrams 8
3.1 Level 1: System Context 8
3.2 Level 2: Containers and Data Store 8
3.3 Level 3: Components 9
3.4 Communication Paths 11
3.5 The OpenAPI Contract 12
4. Team Structure 13
4.1 Team Topologies in Brief 13
4.2 Mapping Modules to Team Types 13
4.3 Interaction Modes 14
4.4 Rationale and Limitations 15
5. Shared Rules 16
5.1 Standard Error Format 16
5.2 Authentication Rules 16
5.3 Common Conventions 17
5.4 Changing the Contract 17
6. Conclusion 18
References 19

List of Figures

Figure 1 Event Storming Board for the Movie Performance Management System 5
Figure 2 System Context Diagram (C4 Level 1) 8
Figure 3 Container Diagram (C4 Level 2) 9
Figure 4 Component Diagram of the Web Application (C4 Level 3) 10
Figure 5 Component Diagram of the API Application (C4 Level 3) 11
Figure 6 Team Topologies Diagram for the Project Team 14

List of Tables

Table 1 Business Glossary 2
Table 2 Technical Glossary 3
Table 3 Domain Events and Their Triggers 6
Table 4 Hot Spots and How They Were Resolved 6
Table 5 Communication Paths 12
Table 6 Endpoint Groups in openapi.yaml 12
Table 7 Team Types and Responsibilities 13
Table 8 Interaction Modes Between Teams 14
Table 9 HTTP Status Codes Used by the API 16
Table 10 Authentication Rules 17
Table 11 Common Conventions 17

1. Problem and Glossary

1.1 Purpose of This Document

This document is the architecture blueprint for the Movie Performance Management System. It is written by the five members of the project team, including Chandani, Avishek, Prakash, Reinaldo and Roshni, so that each person can build their own module based on the same understanding of the problem, the same vocabulary and the same technical contract. It also serves as the group report for submission.

The report covers five areas: the problem and glossary (Section 1), the event storming board (Section 2), the C4 architecture diagrams (Section 3), the team structure based on Team Topologies (Section 4), and the shared rules for errors and authentication (Section 5). The API contract referenced in the report is defined in the openapi.yaml file in the main folder of the Git repository and follows the OpenAPI Specification 3.0 (OpenAPI Initiative, 2020).

1.2 Problem Statement

1.2 Problem Statement

Cinema staff need to know how much money each movie earns every week so that they can decide whether a movie should keep screening. At present, weekly takings are kept in separate files or are visible to only a few people, which makes comparing movies slow and error-prone, and means that a movie earning little may not be noticed until a decision is already late. Because many of the employees who use this information are not technical, it must also be presented in a form that is simple to read. The problem addressed by this project is therefore how to provide ordinary employees with a simple, shared way to enter weekly revenue for each movie, view it by week, and identify which movies need management attention.

1.3 Scope and Intended Users

The system is a prototype that works with dummy data. The following features are in scope:

a) an input page for inserting weekly revenue (dummy data) for a movie;

b) a weekly report page in which the user selects a week and sees the name of each movie and the money it made that week;

c) charts that show revenue by movie and over several weeks, where they help readers;

d) identification of movies that require management review;

e) extending or ending a movie's screening; and

f) login and authentication, which is optional.

Integration with real ticketing or box-office systems, payment handling and revenue forecasting are out of scope. There are two kinds of user. An employee enters data and reads reports. A manager reviews flagged movies and decides whether to extend or end a screening. The end product is a report that analyses the performance of selected movies and that a non-technical employee can read without training.

1.4 Glossary

A shared vocabulary stops the same word from meaning different things in different modules. This follows the idea of a ubiquitous language in domain-driven design (Evans, 2003). Table 1 defines the business terms and Table 2 defines the technical terms used in this report and in `openapi.yaml`.

Table 1

Business Glossary

Term	Definition
Movie	A film that the cinema screens and tracks. It has a title, genre, release date and screening status.
Screening	The period during which a movie is shown at the cinema.
Screening status	The state of a screening: *active*, *extended* or *ended*.
Week (week start)	A reporting week, identified by its Monday date in the form YYYY-MM-DD, for example 2026-10-12.
Revenue	The money a movie made in one week, measured in Australian dollars (AUD).
Performance record	One entry holding the revenue of one movie for one week. There is at most one record per movie per week.
Weekly report	The report for a selected week that lists the name of each movie and the money it made.
Revenue trend	Revenue per movie across several weeks, prepared for line or bar charts.
Review flag	A marker showing that a movie's revenue is below an agreed minimum or has fallen sharply from the previous week.
Management review	The decision step in which a manager considers flagged movies and decides what to do with each screening.
Extend screening	A decision to keep a movie on screen for additional weeks.
End screening	A decision to stop screening a movie.
Employee	A user who enters weekly revenue and reads reports.

Term	Definition
Manager	A user who reviews flagged movies and decides on screenings.

Table 2

Technical Glossary

Term	Definition
API	Application programming interface: the set of requests that one part of the system can make to another.
REST	An architectural style in which resources are identified by URLs and handled with standard HTTP methods (Fielding, 2000).
JSON	A text format for exchanging structured data (Bray, 2017). All request and response bodies use JSON.
OpenAPI	A standard format for describing REST APIs (OpenAPI Initiative, 2020). Here it is the file `openapi.yaml`.
Endpoint	A combination of a URL path and an HTTP method, such as `GET /reports/weekly`.
HTTP status code	A three-digit number in a response that states the outcome, for example 200, 404 or 500 (Fielding et al., 2022).
Bearer token	A string sent in the Authorization header to prove that the caller has logged in (Jones & Hardt, 2012).
Container (C4)	A separately running part of the system, such as a web application, an API application or a database (Brown, n.d.).
Component (C4)	A group of related code inside a container, such as a controller or an API client.
Data store	A container whose job is to keep data, here the relational database.
Stream-aligned team	A team organised around a flow of work that delivers value to users (Skelton & Pais, 2019).
Platform team	A team that provides internal services, such as the API and database, for other teams to use.
Enabling team	A team that helps other teams to gain skills or follow good practice.

Note. Sources for technical terms are cited in the table and listed in the references.

2. Event Storming Board

2.1 Approach

Event storming is a workshop technique in which a business process is mapped on a long timeline of sticky notes, starting from the things that happen (Brandolini, 2013). Domain events are written in the past tense, such as Weekly Revenue Recorded. Around them are placed the commands that cause the events, the people (actors) who issue the commands, the aggregates that hold the data and enforce rules, the policies that trigger automatic reactions, the read models that people look at before deciding, and hot spots that mark open questions or risks.

The technique suits this project because it agrees names before any code is written. The names on the board are the same as those in the glossary (Table 1) and in the endpoints of `openapi.yaml`.

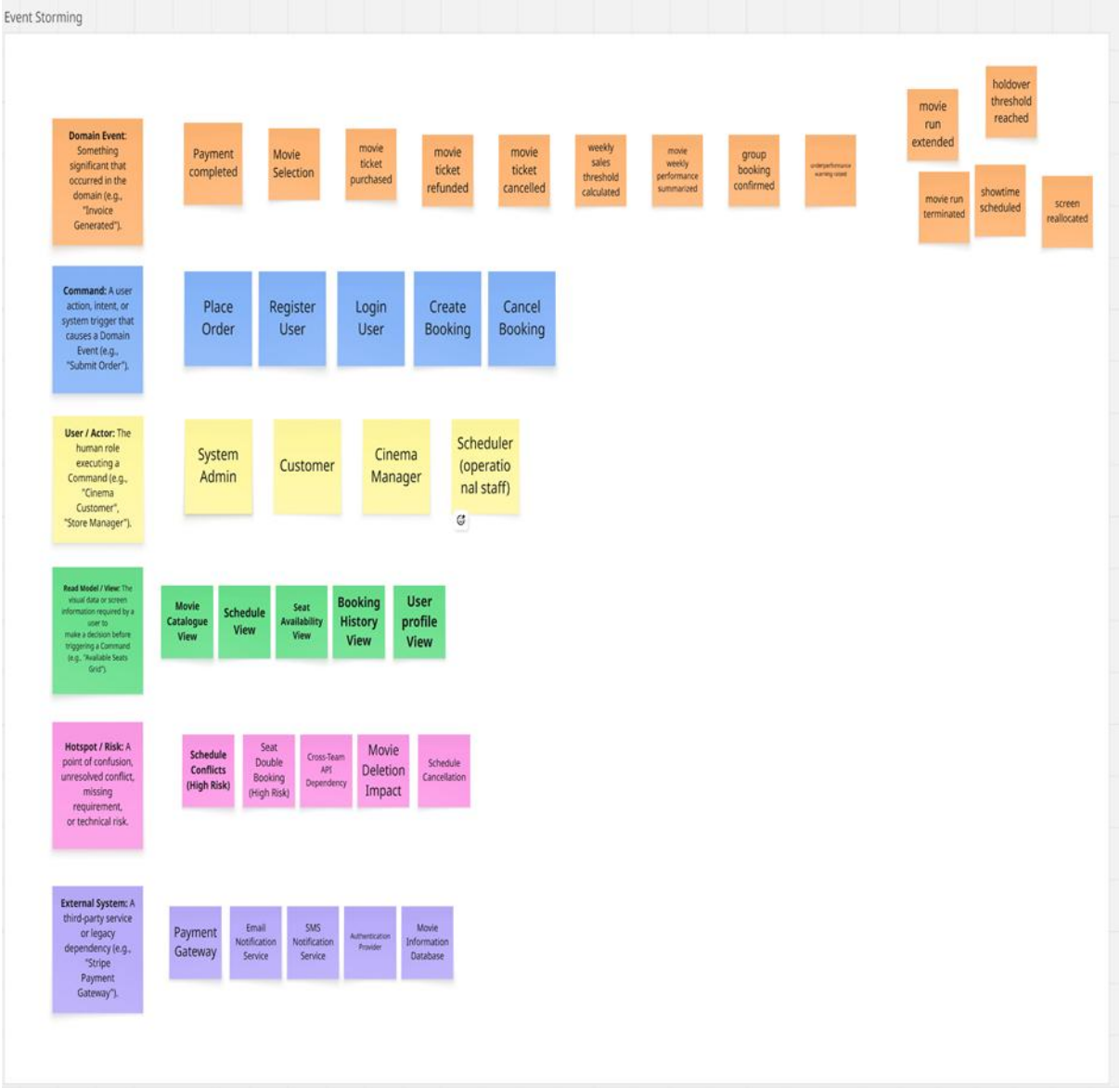
2.2 The Event Storming Board

Figure 1 shows the board. It reads from left to right in two flows. Flow 1 covers setting up movies and recording weekly revenue. A manager adds a movie, and a data entry employee records the revenue of a movie for a given week. A review policy then checks the recorded revenue and flags the movie when it is low or falling.

Flow 2 covers using the information. An employee selects a week and views the weekly report, with a revenue-trend chart as a second read model. A manager looks at the flagged movies and either extends or ends the screening. The optional login flow sits at the end of the board, because the system works without it.

Figure 1

Event Storming Board for the Movie Performance Management System



Note. Notes follow the common event storming colour conventions (Brandolini, 2013). Dashed outlines mark optional features.

2.3 Events, Commands and Aggregates

Table 3 turns the board into a list that developers can build from. Each event is linked to the command that triggers it, the actor, the aggregate and the operation in `openapi.yaml`. The

event Movie Flagged for Review is derived rather than stored: flags are calculated when a manager asks for them for a given week.

Table 3

Domain Events and Their Triggers

Domain event	Command	Actor	Aggregate	API operation
Weekly Revenue Recorded	Record Weekly Revenue	Employee	Performance record	`POST /performance`
Movie Flagged for Review	Apply Review Rule (policy)	System	Performance record	`GET /management/review-flags`

Weekly Report Viewed	Request Weekly Report	Employee	Read model	`GET /reports/weeks`
				`GET /reports/weekly`
				`GET /reports/charts/revenue-trend`

Screening Extended	Extend Screening	Manager	Screening	`POST /movies/{movieId}/screening/extend`
Screening Ended	End Screening	Manager	Screening	`POST /movies/{movieId}/screening/end`
User Logged In (optional)	Log In	Any user	User account	`POST /auth/login`

2.4 Hot Spots and Decisions

Hot spots are the open questions raised on the board. Table 4 records how each one was settled in the API contract, or whether it is still open.

Table 4

Hot Spots and How They Were Resolved

Hot spot	Decision	Status
What counts as a week?	A week is identified by its Monday date (`weekStart`) in ISO 8601 format.	Agreed
Two entries for the same movie and week	One performance record per movie per week. A repeated entry is rejected with 409 Conflict, and a mistake is corrected with `PUT /performance/{recordId}`.	Agreed
When is a movie flagged?	When weekly revenue is below a minimum, or falls by a set percentage from the previous week. The values are passed as `minRevenue` and `dropPercent`, with defaults set by the server.	Rule agreed; values to be confirmed
Is login required?	Login is optional. The bearer-token scheme is defined but not enforced.	Open (optional feature)
Which currency?	All amounts are in Australian dollars (AUD).	Agreed

2.5 Bounded Contexts

The notes fall into four areas of responsibility, which domain-driven design calls bounded contexts (Evans, 2003): Catalogue (movies and their screening status), Performance (weekly performance records), Reporting (the weekly report and chart data) and Management Review (review flags and screening decisions). Authentication is a fifth, optional context. These areas match the groups of endpoints in Table 6 and help divide the work among team members without overlap.

3. C4 Architecture Diagrams

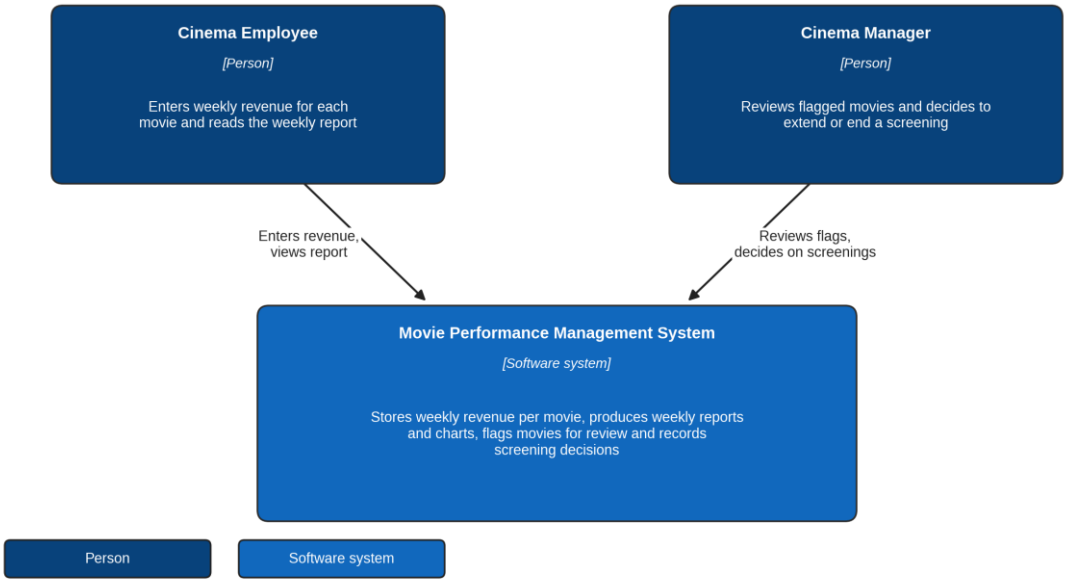
The C4 model describes software architecture at four levels of detail: system context, containers, components and code (Brown, n.d.). This report uses the first three levels, because the repository itself holds the code level. The diagrams use general technology labels such as "Relational database", because what the team has fixed is the contract and the data formats; specific products can be named once they are confirmed.

3.1 Level 1: System Context

Figure 2 shows the system in its environment. A cinema employee enters weekly revenue and reads the weekly report. A cinema manager reviews flagged movies and decides whether to extend or end a screening. No external software systems are connected, because the prototype uses dummy data entered through the input page.

Figure 2

System Context Diagram (C4 Level 1)



Note. Boxes follow the C4 colour scheme: dark blue for people and mid blue for the software system (Brown, n.d.).

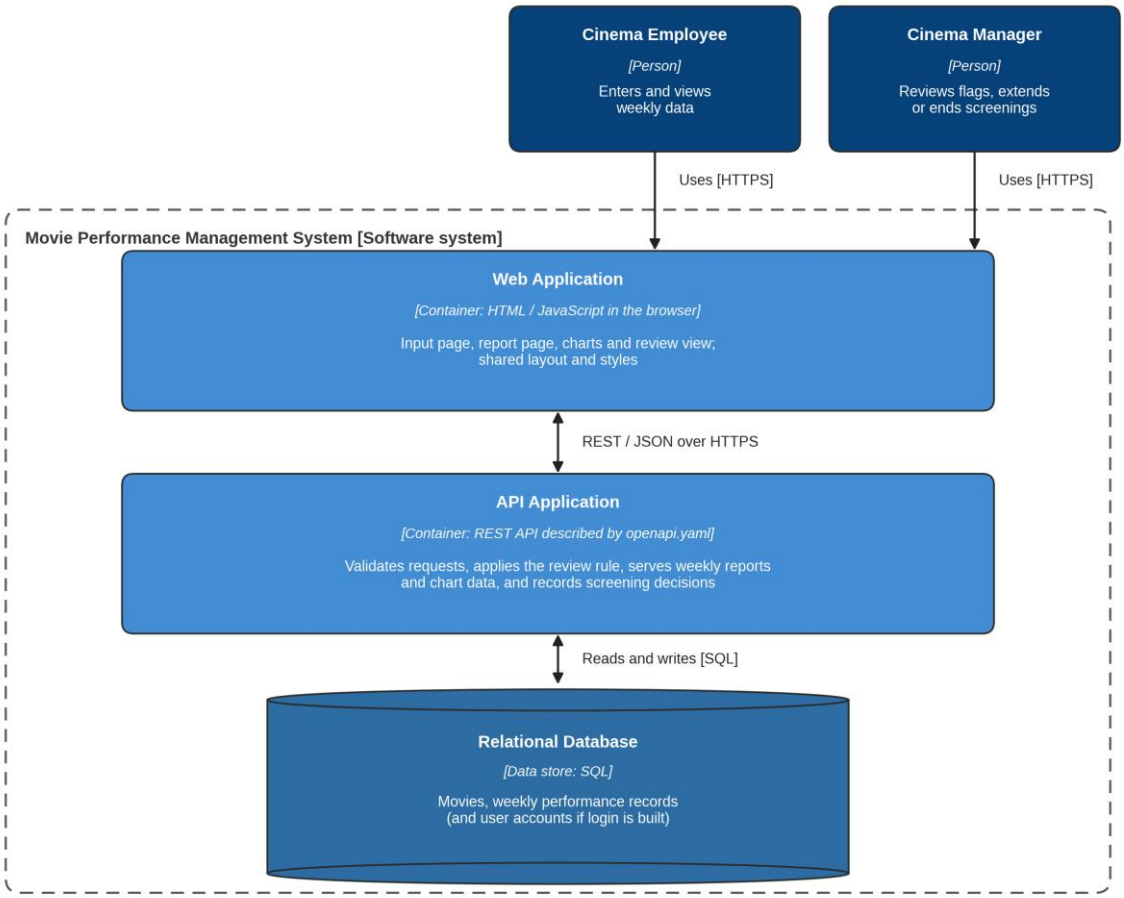
3.2 Level 2: Containers and Data Store

Figure 3 opens the system into three containers. The web application runs in the user's browser and holds the input page, the report page, the charts and the review view. The API application receives REST requests, checks them against the contract, applies the review rule and returns JSON. The relational database is the only data store. It holds movies and weekly performance records, and user accounts if login is built.

Only the API application talks to the database, so every rule about the data is enforced in one place. Communication between the web application and the API application follows the REST style (Fielding, 2000), with requests and responses written in JSON (Bray, 2017). The diagrams show HTTPS because that is the secure setting expected outside local development.

Figure 3

Container Diagram (C4 Level 2)



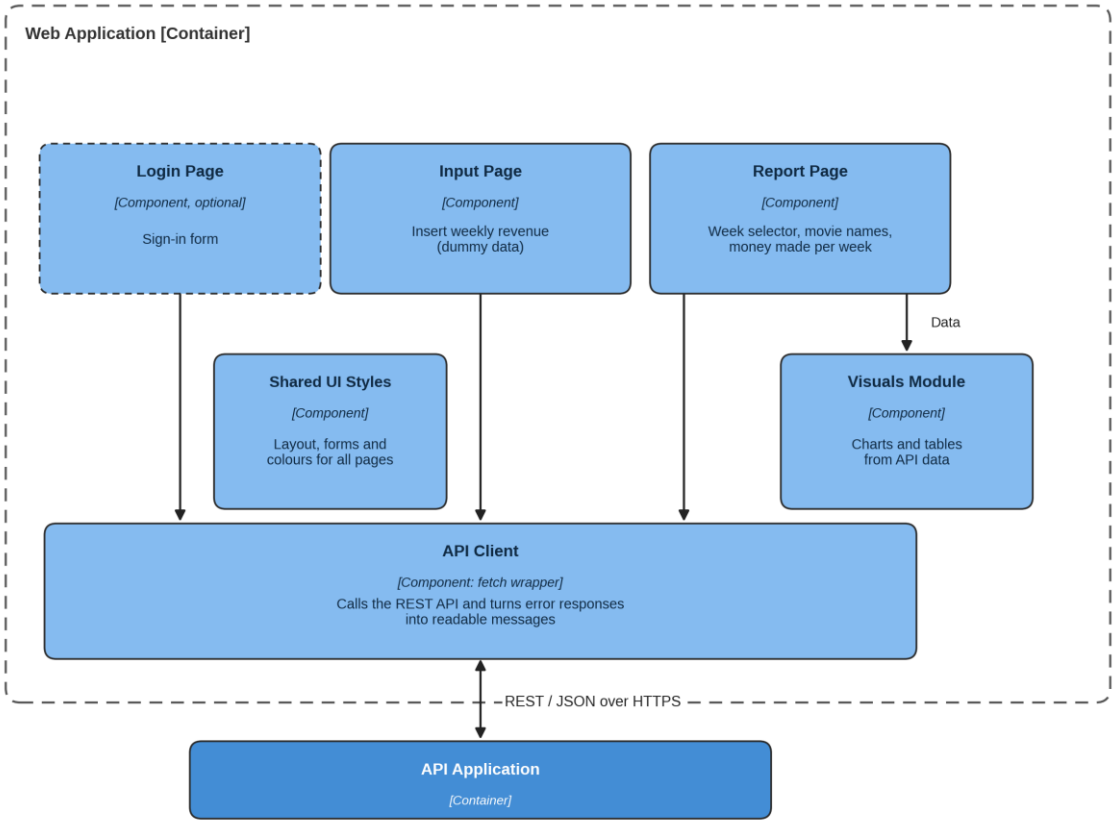
Note. The arrow labelled REST / JSON is the main communication path of the system.

3.3 Level 3: Components

Figure 4 shows the web application. The input page and the report page are the screens that users see. The visuals module receives data from the report page and draws charts and tables. All pages reach the API through one API client component, which sends requests, reads responses and turns error responses into readable messages. Shared UI styles keep the pages consistent and easy to use. The login page is optional.

Figure 4

Component Diagram of the Web Application (C4 Level 3)

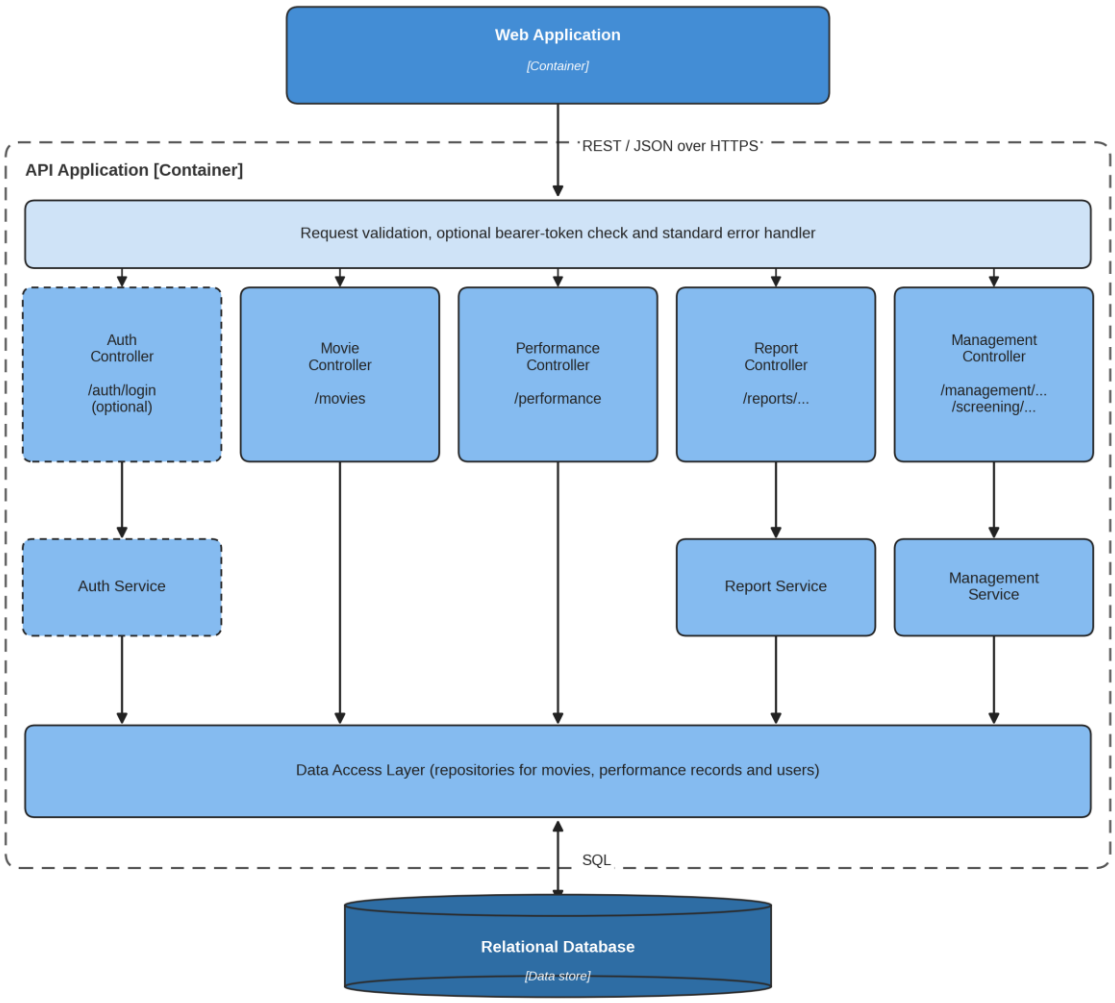


Note. Dashed outlines mark optional parts.

Figure 5 shows the API application. A validation layer checks every request and handles errors in the standard format (Section 5.1). Each group of endpoints has one controller. Services hold the business rules: the review rule and screening decisions in the management service, the weekly summaries in the report service and, if login is built, the authentication service. Only the data access layer talks to the database.

Figure 5

Component Diagram of the API Application (C4 Level 3)



Note. The movie and performance controllers call the data access layer directly because they only create, read, update and delete records.

3.4 Communication Paths

Table 5 lists the communication paths between the elements in Figures 2 to 5. All calls between the web application and the API application use REST with JSON bodies, and every request and response has the shape defined in `openapi.yaml`.

Table 5

Communication Paths

Path	From and to	Protocol and format	Example
P1	Employee or manager to web application	HTTPS in a web browser	Opens the input page or the report page.
P2	Web application to API application	HTTPS, REST, JSON (UTF-8)	`GET /reports/weekly?weekStart=2026-10-12`
P3	API application to database	SQL through a database driver	Select all records for a week; insert a performance record.
P4	API application to web application	HTTP status code with a JSON body	200 with the weekly report, or an error object for 4xx and 5xx.

3.5 The OpenAPI Contract

The team follows an API-first approach: the contract is agreed before the modules are built. The file `openapi.yaml` sits in the main folder of the Git repository, declares version 1.0.0, and describes all paths, methods, parameters, request bodies, response schemas and standard errors. It opens in Swagger Editor without errors. Table 6 groups the operations by module and shows who builds and who uses each group.

Table 6

Endpoint Groups in openapi.yaml

Module	Operations	Built by	Used by
Performance	`POST /performance` `PUT, DELETE /performance/{recordId}`	Chandani (API), Rose (data)	Input page (Prakash)

Reports	`GET /reports/weeks`
	`GET /reports/weekly`
	`GET /reports/charts/revenue-trend`
		Chandani (API), Rose (data)	Report page (Prakash), charts (Avishek)

Management	`GET /management/review-flags`
	`POST /movies/{movieId}/screening/extend`
	`POST /movies/{movieId}/screening/end`
		Chandani (API), Rose (data)	Report page (Prakash), review view (Avishek)

Auth (optional)	`POST /auth/login`	Chandani (API)	All pages

4. Team Structure

4.1 Team Topologies in Brief

Team Topologies proposes four basic team types and three interaction modes for organising teams so that work flows quickly (Skelton & Pais, 2019). A stream-aligned team is organised around a flow of work and delivers value to users. A platform team provides internal services that make stream-aligned teams more productive. An enabling team helps other teams to gain missing skills or follow good practice. A complicated-subsystem team looks after a part of the system that needs specialist knowledge.

Teams interact in one of three ways. In collaboration they work closely together. In X-as-a-Service one team uses what another provides with very little coordination. In facilitating one team helps another to learn or improve. Designing teams together with the architecture matters because of Conway's law: the structure of a system tends to copy the communication structure of the group that builds it (Conway, 1968).

4.2 Mapping Modules to Team Types

The project has five members. Table 7 proposes how their roles map to the team types, and Figure 6 shows the result. Because the group is small, one person may hold more than one role, so the types describe responsibilities rather than separate departments.

Table 7

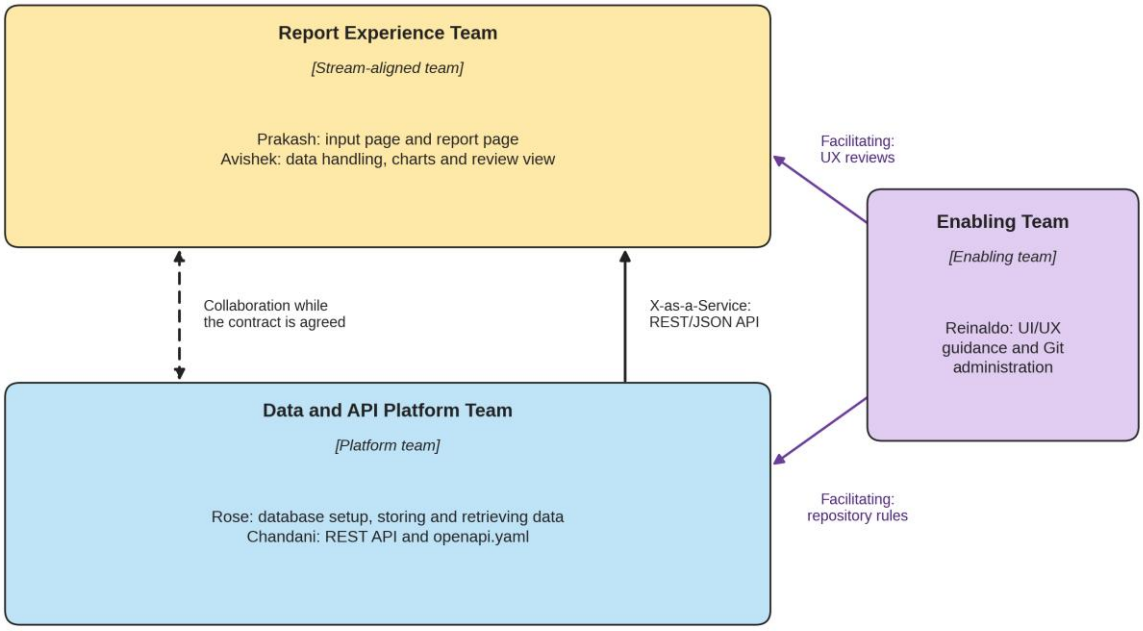
Team Types and Responsibilities

Team type	Members	Responsibilities	Owned parts
Stream-aligned: Report Experience Team	Prakash, Avishek	Deliver the user-facing flow from entering data to reading the weekly report, including charts and the view of movies that need review.	Input page, report page, data handling and charts
Platform: Data and API Platform Team	Rose, Chandani	Provide data storage and the REST API as a service that the stream-aligned team can use.	Database, API application, `openapi.yaml` implementation
Enabling team	Reinaldo	Give usability and design guidance. Administer the Git repository: access, branch rules and merges into the main branch.	UI/UX design, repository setup

Team type	Members	Responsibilities	Owned parts
Complicated-subsystem	None	No part of the system needs specialist knowledge. The review rule is a simple comparison.	Not applicable

Figure 6

Team Topologies Diagram for the Project Team



Note. Solid arrows show X-as-a-Service and facilitating; the dashed arrow shows collaboration.

4.3 Interaction Modes

Table 8 describes how the teams work with each other. Most of the time the report team uses the API as a service. Close collaboration is limited to the period in which the contract is being agreed.

Table 8

Interaction Modes Between Teams

Between	Mode	How it works
Report Experience Team and Data and API Platform Team	X-as-a-Service	The report team calls the API as described in `openapi.yaml` and does not need to know how the database is built. Pages can be built against the examples in the file.
Report Experience Team and Data and API Platform Team	Collaboration	While the contract is being agreed, both teams work closely and change `openapi.yaml` by pull request.
Enabling team to Report Experience Team	Facilitating	Reinaldo reviews pages for ease of use and shares layout and style guidance.
Enabling team to Data and API Platform Team	Facilitating	As Git administrator, Reinaldo sets repository rules and supports reviews and merges.

4.4 Rationale and Limitations

The mapping keeps dependencies mostly one-way. The report team depends on the API, but the platform team does not depend on the pages. This lets the pages be built against mock data while the API is still being written.

There are two limitations. First, the platform team could become a bottleneck, because every new data need must pass through it. A shared contract and collaboration during its design reduce this risk. Second, the enabling role is held by one person who also carries UI/UX work, so repository administration should be treated as a shared duty that others can cover if that person is unavailable.

5. Shared Rules

5.1 Standard Error Format

All errors use the same JSON object, defined as the `Error` schema in `openapi.yaml`, so that the web application can handle every failure in one place. The object has two fields: `code`, a short machine-readable string, and `message`, a sentence that a person can read. HTTP status codes keep their standard meanings (Fielding et al., 2022). For example, a request for a movie that does not exist returns status 404 with this body:

```
{
  "code": "NOT_FOUND",
  "message": "Movie 42 was not found."
}
```

Messages must not reveal internal details such as database queries or stack traces. These are written to the server log instead (OWASP Foundation, 2023). Table 9 lists the status codes used by the API.

Table 9

HTTP Status Codes Used by the API

Status	code value	Meaning	Used when
200 OK	Not applicable	The request succeeded.	A read, update or action returns data.
201 Created	Not applicable	A new resource was created.	`POST /movies`, `POST /performance`
204 No Content	Not applicable	The resource was deleted.	Any `DELETE` operation
400 Bad Request	BAD_REQUEST	The input is missing or invalid.	A date has the wrong format, revenue is negative, or weekStart is missing.
401 Unauthorized	UNAUTHORIZED	The token is missing or invalid, or the login failed.	Only when login is enabled.
404 Not Found	NOT_FOUND	The movie or record does not exist.	An unknown `movieId` or recordId.
409 Conflict	CONFLICT	The request clashes with the current state.	A second record for the same movie and week, or extending a screening that has ended.

Status	code value	Meaning	Used when
500 Internal Server Error	SERVER_ERROR	An unexpected failure on the server.	Any endpoint.

5.2 Authentication Rules

Login is an optional feature. The contract already defines the rules, so the pages can be built to support it without a redesign. Table 10 sets out the rules.

Table 10

Authentication Rules

Rule	Description
Optional by default	The root `security` setting in `openapi.yaml` allows requests without a token. If login is built, the empty option is removed so that every endpoint except login requires a token.
Login	`POST /auth/login` accepts a username and a password in a JSON body and returns a `token` and `expiresIn` (seconds).
Sending the token	The token is sent in the Authorization header as `Bearer <token>` on every request except login (Jones & Hardt, 2012).
Failure	A missing, invalid or expired token returns 401 with the standard error object.
Secrets	Passwords are never stored in plain text or written to logs. Deployments outside local development use HTTPS (OWASP Foundation, 2023).
Roles (proposed)	If roles are added, employees may enter data and read reports, and only managers may extend or end screenings. This is not part of version 1.0.0 of the contract and would add a 403 Forbidden response.

5.3 Common Conventions

Table 11 lists the conventions that every module follows.

Table 11

Common Conventions

Convention	Rule
Base path	All endpoints sit under `/api/v1`.
Data format	Request and response bodies are JSON in UTF-8 (Bray, 2017).
Dates	ISO 8601 format, YYYY-MM-DD. A week is identified by its Monday (`weekStart`).
Money	Non-negative numbers in AUD, with up to two decimal places.
Names and identifiers	Field names use camelCase, such as `movieId` and `weekStart`. Identifiers are whole numbers.

5.4 Changing the Contract

Any change to `openapi.yaml` is made by pull request and reviewed by the affected teammates before the Git administrator merges it into the main branch. A change that could break existing callers, such as removing a field or renaming a path, raises the major version number; a compatible addition raises the minor version number. The file must still open in Swagger Editor with no errors after every change.

6. Conclusion

This blueprint gives the team a shared base for building the Movie Performance Management System. The problem statement and glossary fix the language, the event storming board links that language to events, commands and endpoints, and the C4 diagrams show how the web application, the API application and the database communicate through REST and JSON.

The Team Topologies mapping separates a stream-aligned report team from a platform team that provides data and the API, with an enabling role that supports both. The shared rules for errors, authentication and conventions mean that each module can be built independently and still fit with the others. Two matters remain open: the exact review thresholds, and whether the optional login feature will be built.

References

Brandolini, A. (2013). *Introducing EventStorming*. Leanpub.

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc8259

Brown, S. (n.d.). *The C4 model for visualising software architecture*. Retrieved October 19, 2026, from https://c4model.com

Conway, M. E. (1968). How do committees invent? *Datamation, 14*(4), 28–31.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R. T., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc9110

Jones, M., & Hardt, D. (2012). *The OAuth 2.0 authorization framework: Bearer token usage* (RFC 6750). Internet Engineering Task Force. https://www.rfc-editor.org/info/rfc6750

OpenAPI Initiative. (2020). *OpenAPI specification* (Version 3.0.3). https://spec.openapis.org/oas/v3.0.3

OWASP Foundation. (2023). *OWASP API security top 10: 2023*. https://owasp.org/API-Security/editions/2023/en/0x00-header/

Skelton, M., & Pais, M. (2019). *Team Topologies: Organizing business and technology teams for fast flow*. IT Revolution Press.